

tf.compat.v1.nn.ctc_loss Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/nn/ctc_loss

Computes the CTC (Connectionist Temporal Classification) Loss.

Function Signatures

```
tf.compat.v1.nn.ctc_loss( labels, inputs=None,
sequence_length=None, preprocess_collapse_repeated=False,
ctc_merge_repeated=True,
ignore_longer_outputs_than_inputs=False, time_major=True,
logits=None )

sequence_length(b) <= time for all b
max(labels.indices(labels.indices[:, 1] == b, 2)) <=
sequence_length(b) for all b.
```

Code Examples

```
tf.compat.v1.nn.ctc_loss(  
    labels,  
    inputs=None,  
    sequence_length=None,  
    preprocess_collapse_repeated=False,  
    ctc_merge_repeated=True,  
    ignore_longer_outputs_than_inputs=False,  
    time_major=True,  
    logits=None  
)
```

```
tf.compat.v1.nn.ctc_loss(  
    labels,  
    inputs=None,  
    sequence_length=None,  
    preprocess_collapse_repeated=False,  
    ctc_merge_repeated=True,  
    ignore_longer_outputs_than_inputs=False,  
    time_major=True,  
    logits=None  
)
```

```
sequence_length(b) <= time for all b  
max(labels.indices(labels.indices[:, 1] == b, 2))  
<= sequence_length(b) for all b.
```

```
sequence_length(b) <= time for all b  
max(labels.indices(labels.indices[:, 1] == b, 2))  
<= sequence_length(b) for all b.
```

Documentation

Computes the CTC (Connectionist Temporal Classification) Loss.

This op implements the CTC loss as presented in (Graves et al., 2006).

Input requirements:

Notes:

This class performs the softmax operation for you, so inputs should be e.g. linear projections of outputs by an LSTM.

The inputs Tensor's innermost dimension size, `num_classes`, represents `num_labels + 1` classes, where `num_labels` is the number of true labels, and the largest value (`num_classes - 1`) is reserved for the blank label.

For example, for a vocabulary containing 3 labels [a, b, c], `num_classes = 4` and the labels indexing is {a: 0, b: 1, c: 2, blank: 3}.

Regarding the arguments `preprocess_collapse_repeated` and `ctc_merge_repeated`:

If `preprocess_collapse_repeated` is `True`, then a preprocessing step runs before loss calculation, wherein repeated labels passed to the loss are merged into single labels. This is useful if the training labels come from, e.g., forced alignments and therefore have unnecessary repetitions.

If `ctc_merge_repeated` is set `False`, then deep within the CTC calculation, repeated non-blank labels will not be merged and are interpreted as individual labels. This is a simplified (non-standard) version of CTC.

Here is a table of the (roughly) expected first order behavior:

- preprocess_collapse_repeated=False, ctc_merge_repeated=True

Classical CTC behavior: Outputs true repeated classes with blanks in between, and can also output repeated classes with no blanks in between that need to be collapsed by the decoder.

- preprocess_collapse_repeated=True, ctc_merge_repeated=False

Never learns to output repeated classes, as they are collapsed in the input labels before training.

- preprocess_collapse_repeated=False, ctc_merge_repeated=False

Outputs repeated classes with blanks in between, but generally does not require the decoder to collapse/merge repeated classes.

- preprocess_collapse_repeated=True, ctc_merge_repeated=True

Untested. Very likely will not learn to output repeated classes.

preprocess_collapse_repeated=False, ctc_merge_repeated=True

Classical CTC behavior: Outputs true repeated classes with blanks in between, and can also output repeated classes with no blanks in between that need to be collapsed by the decoder.

preprocess_collapse_repeated=True, ctc_merge_repeated=False

Never learns to output repeated classes, as they are collapsed in the input labels before training.

`preprocess_collapse_repeated=False, ctc_merge_repeated=False`

Outputs repeated classes with blanks in between, but generally does not require the decoder to collapse/merge repeated classes.

`preprocess_collapse_repeated=True, ctc_merge_repeated=True`

Untested. Very likely will not learn to output repeated classes.

The `ignore_longer_outputs_than_inputs` option allows to specify the behavior of the `CTCLoss` when dealing with sequences that have longer outputs than inputs. If true, the `CTCLoss` will simply return zero gradient for those items, otherwise an `InvalidArgument` error is returned, stopping training.

Returns

`## Raises`

`## References`